Interacció i Disseny d'Interfícies

Bloc 1: Introducció IDI

Creats per: Stefan Ivaylov Patchaliev

Índex

1. Processos de visualització 3
1.1. Versions recomanades per treballar el lab a casa: 3
1.2. Models i visualitzacions: 3
1.3 Ús dels VAOs i VBOs: 4
1.4 Procés de visualització complet: 5
1.5 Sistemes de coordenades (SC): 5
2. Transformacions geomètriques 6
2.1 Possibles transformacions: 6
2.2 Com aplicar-les en el codi: 8
2.3 Capses de models: 8

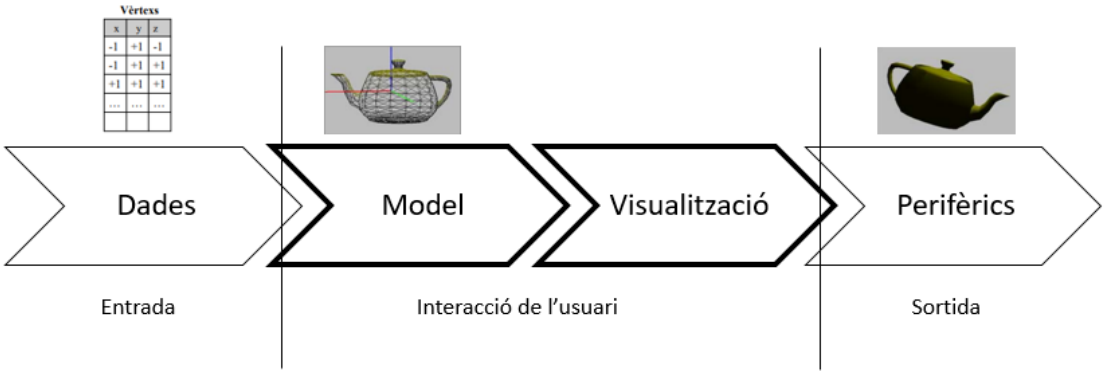
1. Processos de visualització

1.1. Versions recomanades per treballar el lab a casa:

- **OpenGL**: Aplicació per la creació de gràfics en 3D (versió 3.3 o superior).
- **Qt**: Aplicació per la creació de programes perquè l'usuari interactuï amb l'entorn gràfic (versió recomanada 5.x).
- **Ubuntu**: Idealment versions del 18 al 20, la 21 també funciona però té problemes.

1.2. Models i visualitzacions:

Primerament hem de tenir en compte l'ordre per entendre el funcionament, tot seguit tenim un esquema:

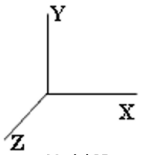


- Les **dades** són tot aquella informació que posem per crear el nostre model.
- El **model** és creat a partir de les dades, en el centre de les coordenades. L'usuari pot fer modificacions sobre el model (escalar, rotar, transposar...)
- La **visualització** són les diferents modificacions que li fa l'usuari per veure l'objecte (llums, càmera..)
- Els **perifèrics** seria el resultat final que veurem per pantalla, un cop passat per tots els passos.

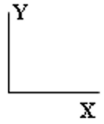
És molt important a tenir en compte la manera en la que s'introdueixen les dades, ja que sempre introduirem **les dades dels models a partir de triangles**, és a dir, 3 grups de coordenades x, y, z.

Inicialment començarem amb dibuixos en 2D, per tant podrem "ignorar" la coordenada z, que serà la responsable de donar-nos profunditat en el model.

Recordem la que representem els nostres models en la **finestra gràfica en 2D**.

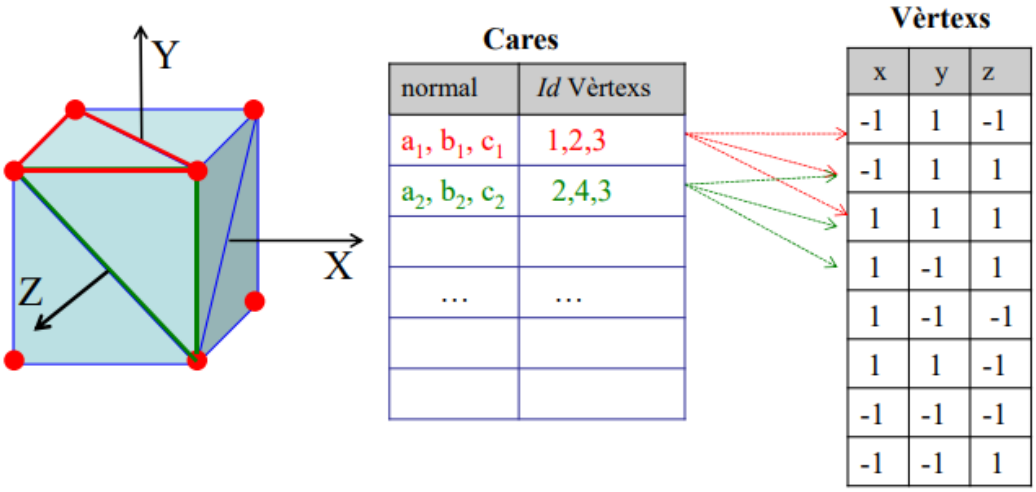


Model 3D



Finestra gràfica

Per exemple, si volem representar un cub, cal dividir el cub en triangles per evitar repetir vèrtexs innecessaris i per poder representar-lo en grups de 3 coordenades:



Un cop creat el nostre model el guardarem en un buffer anomenat **VAO**, utilitzat per **encapsular les dades** del model. La idea del VAO és aprofitar el model creat per evitar crear un altre model per cada una de les copies que necessitem en la nostra imatge.

Es crea un **únic VAO per model**, si hem de fer canvis de posició, mida, etc., es faran aplicant transformacions geomètriques.

Apart dels VAOs també hi els VBO. Els VBO són buffers on es guarden les coordenades del model, els colors del model, les normals del model... Tot i que són opcionals a excepció de les coordenades (per tant ens cal mínim un VBO per cada un del VAOs existents).

A més, ens cal una **validació del model**. Haurem de comprovar 3 punts:

1. Totes les cares han d'estar orientades correctament (cal mirar la normal).
2. Tots els vèrtexs han d'estar ordenats coherentment
3. Les arestes, en un model tancat, separen dos triangles/cares.

Anomenem **escena** a la imatge resultat que veiem per la finestra gràfica, amb tots els objectes que haguem inserit.

1.3 Ús dels VAOs i VBOs:

Per utilitzar els VAOs i VBOs que hem creat, caldrà posar-ho en el codi:

```
void pinta_cub() {
        glBindVertexArray (VAO);        //envia el VAO
        modelTransform(TG);             //fa els canvis sobre el VAO
        modelMatrix(TG);                //envia uniform
        pintaModel();                   //pinta l'escena final
}
```

1.4 Procés de visualització complet:

1. Processat de vèrtex:
Rep les coordenades del vèrtex SCM (TG) i el transforma a SCC (clipping), passant primer per SCA (VM) i SCO (PM).

2. Clipping + Perspective division:
Passem les coordenades a SCN. S'eliminen tots els triangles que es trobin sencers fora del rang de visió.

3. Device transform:
Passem les coordenades a SCD. Es té en compte el viewport i la relació d'aspecte (ra).

4. Back-face culling:
És una opció activable/desactivable. Al estar activat, elimina les parts amagades dels triangles, és a dir, té en compte les normals. Només elimina si és 100% segur que no es veurà.

5. Rasterització:
Rep els triangles i els pinta amb punts, segments o polígons. S'envia després al FS.

6. Processat de fragments:
És feta en el Fragment Shader, on es posen els colors, textures...

7. Z-Buffer/Depth-buffer:
És una opció activable/desactivable. Al estar activat, elimina parts amagades dels triangles per altres triangles. No té en compte el back-face culling.

1.5 Sistemes de coordenades (SC):

- **SC Model**: Coordenades inicials pels models (I).

- **SC Aplicació**: Obtingudes un cop les SCM són posicionades a l'escena (TG*I).

- **SC Observador**: Obtingudes un cop les SCA són transformades segons les coordenades de l'observador (VM*TG*I).

- **SC Clipping**: Obtingudes un cop les SCO són transformades segons l'òptica de la càmera (V=PM*VM*TG*I).

- **SC Normalitzades**: Obtingudes del resultat del clipping (V=V/w).

- **SC Dispositiu**: Obtingudes del resultat del Device Transform per ocupar la finestra gràfica correctament (X*V, sent X la transformació per ocupar tota la pantalla).

- **SC Fragment**: Obtingudes després de la rasterització, sent les coordenades de cada fragment de la finestra.

2. Transformacions geomètriques

En aquest tema tractarem la manera que té l'usuari d'interactuar amb els diferents models creats prèviament i com crear diferents versions del mateix model (a partir del ús dels VAOs).

La millor manera d'interactuar és a partir de les **transformacions geomètriques (TG)**, matrius de 4x4 que utilitzem per modificar el VAO creat i així crear l'escena demanada. Cada model/objecte tindrà la seva pròpia TG.

Cal tenir en compte que els nostres models són 3D mentre que la TG serà de 4x4, per tant ens caldrà afegir un valor extra (que posarem a 1).

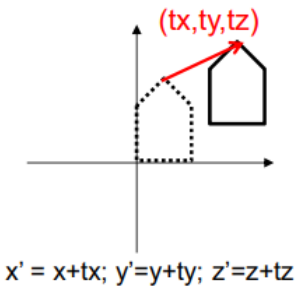
2.1 Possibles transformacions:

Translació: Serveix per moure el nostre objecte a una nova posició de l'escena. Es mou l'objecte sencer, per tant només cal fixar-s'hi en un dels vèrtexs i buscar on es troba la posició final.

El codi a utilitzar és: "TG = translate(mat4(TG), *moviment x*, *moviment y*, *moviment z*)";

En pseudo-codi és escrit de la següent manera: "TG = translate(TG, 2, 1, 0);"
En aquest cas, el moviment seria augmentar 2 en l'eix x, augmentar 1 en l'eix y i no fer cap canvi en el eix z, com podem veure en la imatge següent.

Important tenir en compte que els números que indiquem en el codi indiquen **el moviment a fer**, no la posició a ocupar!

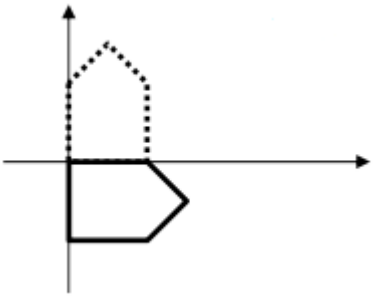


Rotació: Serveix per rotar el nostre objecte de l'escena. En aquest cas, cal tenir en compte que **el gir es fa en l'origen de coordenades**. Al fer el gir, es veurà afectada "tota l'escena".

El codi a utilitzar és: "TG = rotate(mat4(TG), *float(angle)*, *vec3(eix rotació)*)";
El primer paràmetre és la TG a utilitzar, el segon és l'angle de gir (sempre en radians!!) i el tercer és l'eix segons el que farem el gir.

En pseudo-codi és escrit de la següent manera: "TG = rotate(TG, float(-π/2), vec3(0,0,1))";
En aquest cas rotem el nostre objecte 90º horàriament, al voltant de l'eix z, com veiem a la imatge següent.

Important! Si volem girar un objecte però no moure'l de lloc, haurem de portar-lo primer al centre de coordenades, fer el gir i finalment retornar-lo a lloc.

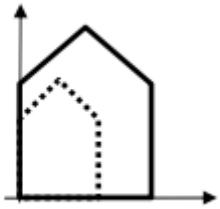


Escalar: Serveix per escalar el nostre objecte de l'escena. En aquest cas, igual que en el gir, cal tenir en compte que **l'escalada es fa segons l'origen de coordenades**.

El codi a utilitzar és: "TG = scale(mat4(TG), *vec3(escala)*)";
El primer paràmetre és la TG a utilitzar i el segon és per indicar com volem escalar el nostre objecte (al ser tres valors, un per cada eix, podem escalar de manera uniforme o no uniforme).

En pseudo-codi és escrit de la següent manera: "TG = scale(TG, vec3(2,1.5,0))";
En aquest cas podem veure que l'escalat no és uniforme, augmentant més la x que la y.

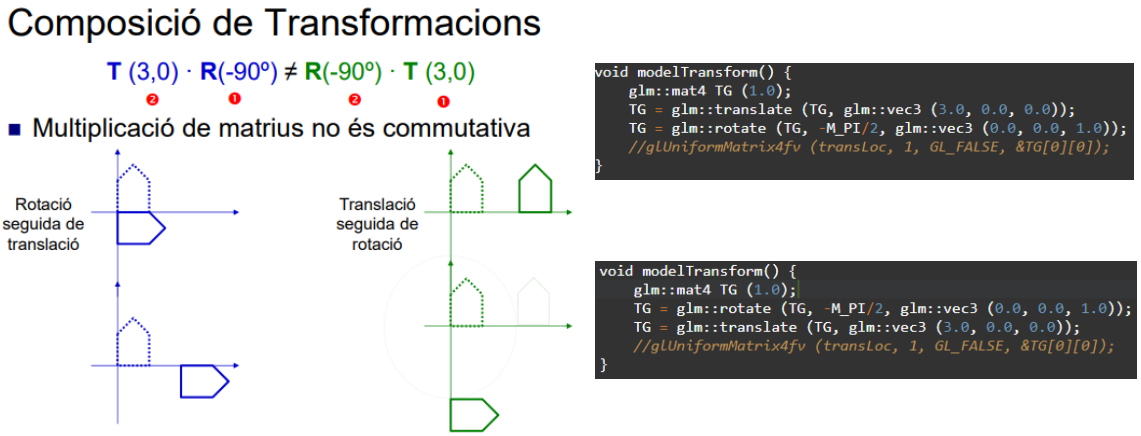
Important! Igual que en el cas de la rotació, hem de portar-lo al centre de coordenades per escalar l'objecte.



2.2 Com aplicar-les en el codi:

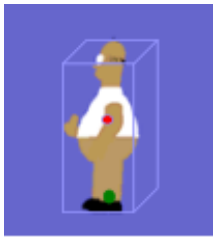
Podem encadenar totes les transformacions que volem fer-li a un objecte en un mateix codi, però hi ha certes coses que haurem de tenir present a l'hora de crear el codi:

1. El codi es llegeix de baix a dalt o de dreta a esquerra (sí, al revés).
2. Primerament (de manera general) haurem de portar l'objecte a l'origen de coordenades.
3. L'ordre **IMPORTA** al fer el nostre codi



2.3 Capses de models:

Generalment els objectes són irregulars per tant, per poder interactuar amb ells, els introduïm en **una capsa amb tot l'objecte dins** (l'exemple típic, els patricios). Els límits de la capsa contenidora són formats pels extrems del model.

Dades importants:

- Límits de la capsa: Pmin(Xmin, Ymin, Zmin), Pmax(Xmax, Ymax, Zmax)
- **Amplada** = Xmax-Xmin; **Alçada** = Ymax – Ymin; **Profunditat** = Zmax-Zmin;
- Centre capsa: C = ((Xmax+Xmin)/2, (Ymax+Ymin)/2, (Zmax+Zmin)/2);
- Base capsa: B = ((Xmax+Xmin)/2, Ymin, (Zmax+Zmin)/2);